TRƯỜNG ĐẠI HỌC PHENIKAA
Khoa/Bộ môn: Điện-điện tử

BÁO CÁO BÀI KIỂM TRA SỐ 2

Thiết kế số nhỏ và quy trình RTL-to-GDSII bằng công cụ nguồn mở

Giảng viên hướng dẫn: Đinh Văn Nam

Học phần	EEE703128 — Nhập môn Thiết kế Vi mạch Tích hợp
Mã học phần	EEE703128 · 3 tín chỉ 3(3;0;6)
Tên bài	Bài kiểm tra số 2 — Báo cáo cá nhân RTL-to-GDSII
Mã đề · Tên đề tài	F10 — FSM bắt tay bốn pha req/ack
Họ và tên sinh viên	Lê Chí Hoàng Cường
Mã số sinh viên	24101280
Lớp	Nhập môn thiết kế vi mạch tích hợp-1-1-26(N01)
Giảng viên hướng dẫn	Đinh Văn Nam
Kho GitHub	https://github.com/lechihoangcuong/eee703128-f10-24101280
Commit mã nguồn nộp bài	7ae2df4 (RTL/TB)
Lần chạy CI dùng cho báo cáo	Run 40179312464 · commit thực thi b59c838
Ngày nộp	26/10/2026

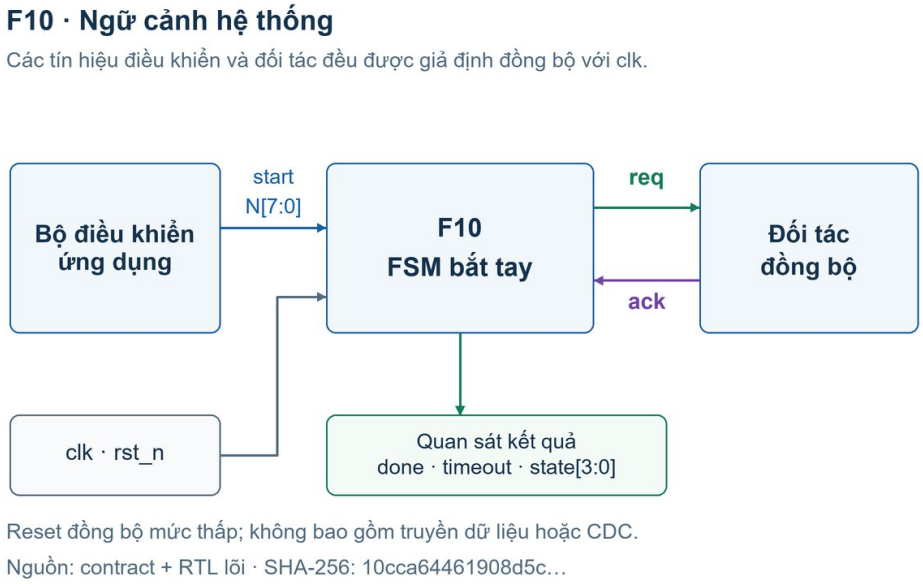
Số liệu CI ngày 25/10/2026 · Commit thực thi b59c838 · Chi tiết tái lập ở Phụ lục C

Mục lục

1. Đặt vấn đề và đặc tả 3
2. Thiết kế RTL 5
3. Kiểm chứng chức năng 7
4. Kết quả tổng hợp 11
5. Kết quả phân tích thời gian 12
6. Công suất và kiểm tra vật lý 13
7. Quan sát layout 14
8. Trả lời câu phân tích riêng của đề 15
9. Kết luận, giới hạn và tự đánh giá 16
Phụ lục A — Mã nguồn RTL đầy đủ 17
Phụ lục B — Testbench đầy đủ 22
Phụ lục C — Thông tin tái lập 37
Cam kết 38

1. Đặt vấn đề và đặc tả

F10 là bộ điều khiển bên yêu cầu (requester) cho giao thức bắt tay bốn pha: REQ↑ → ACK↑ → REQ↓ → ACK↓. Khi đủ bốn pha, mạch phát DONE một chu kỳ; nếu đối tác không phản hồi đúng hạn ở một pha chờ, mạch phát TIMEOUT và trở về IDLE. Mạch điều khiển giao thức, không truyền payload. [S1, S2]



Hình 1. Vị trí của F10; đối tác phát ACK được mô hình hóa trong testbench.

Đây là mạch số tuần tự: state, bộ đếm và ngưỡng đã chốt lưu thông tin qua cạnh clock. Đối tác dùng cùng miền clock; CDC và metastability ngoài phạm vi. Cùng clock chưa đủ bảo đảm đối tác tuân thủ đúng bốn pha. [S2]

Giao diện và ánh xạ chân

Bảng 1.1 — Bảng tín hiệu vào/ra

Tín hiệu	Chân	Độ rộng	Chiều	Mức tích cực	Ý nghĩa
clk	clk	1	vào	cạnh lên	Clock cập nhật thanh ghi
rst_n	rst_n	1	vào	0	Reset đồng bộ
ena	ena	1	vào	1	Cho nhận START mới; không đóng băng giao dịch đang chạy
timeout_value	ui_in[7:0]	8	vào	—	Ngưỡng N không dấu, 0–255
ack	uio_in[0]	1	vào	1	Phản hồi từ đối tác, theo charter
start	uio_in[1]	1	vào	1	Đề xuất tích hợp; charter chưa gán chân START
req	uo_out[0]	1	ra	1	Yêu cầu; ngõ ra Moore từ state
done	uo_out[1]	1	ra	1	Xung hoàn tất một chu kỳ
timeout	uo_out[2]	1	ra	1	Xung hết hạn một chu kỳ
state	uo_out[6:3]	4	ra	—	Quan sát mã trạng thái 0, 1, 2, 3
không dùng	uo_out[7]	1	ra	—	Cố định 0
uio_out / uio_oe	uio_out / uio_oe	8 / 8	ra	—	Đều 0; toàn bộ uio ở chế độ đầu vào

Bảng 1.2 — Giả định và trường hợp biên

#	Giả định / trường hợp biên	Xử lý theo đặc tả
1	Reset tích cực thấp, đồng bộ	Chỉ xóa thanh ghi tại cạnh lên clk khi rst_n=0; ưu tiên cao nhất.
2	N=0; N=1; N=255 (ca biên)	Chờ tối đa max(1,N) cạnh cho mỗi pha; ACK hợp lệ tại deadline được ưu tiên.
3	START khi bận / ACK cũ đang cao	Không chen hoặc xếp hàng giao dịch; chỉ bắt đầu khi IDLE, start=1, ack=0.
4	Thay đổi N trong lúc bận	Dùng timeout_q đã chốt; thay input không đổi deadline đang chạy.
5	ACK không lên hoặc không hạ	TIMEOUT một chu kỳ rồi về IDLE; requester không tự sửa trạng thái đối tác.

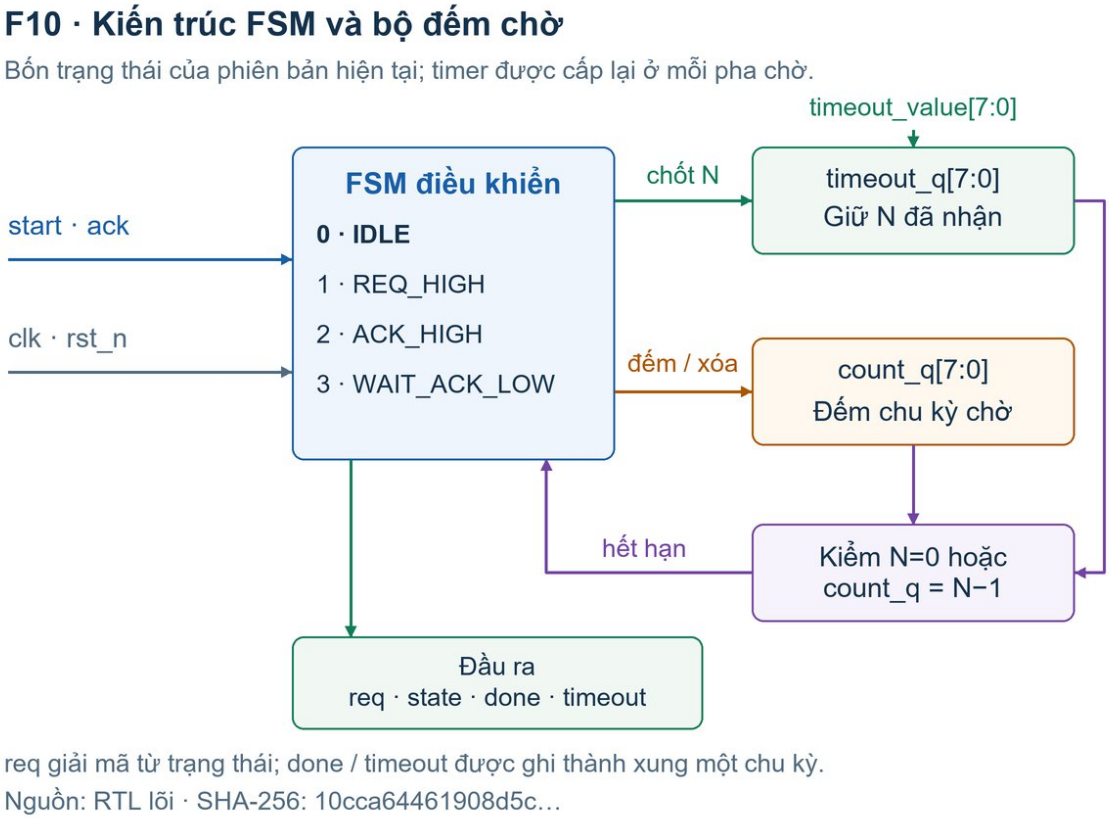
Wrapper nhận start nội bộ = ena AND uio_in[1]. Tổng số bit có chức năng: 10 bit vào và 7 bit ra, trong ngân sách 24 chân. Việc chọn uio_in[1] là phương án tích hợp cần giảng viên xác nhận, không phải ánh xạ START đã được phê duyệt. [E2]

Lõi hiện có là requester; responder nằm trong môi trường testbench. Hồ sơ này không nhận đã triển khai hai khối RTL requester/responder độc lập. Khi START giữ mức 1 tới lúc mạch trở về IDLE và ACK=0, một giao dịch mới có thể được nhận. [S2, E1]

Nguồn và thay đổi: lõi f10_handshake_controller.sv giữ nguyên byte từ ZIP đã cung cấp. Wrapper, tên top triển khai, testbench và script lưu bằng chứng được bổ sung; ghi chi tiết tại docs/PROVENANCE_AND_CHANGES.md. Phụ lục A và B chứa toàn bộ mã thực tế. [S2]

2. Thiết kế RTL

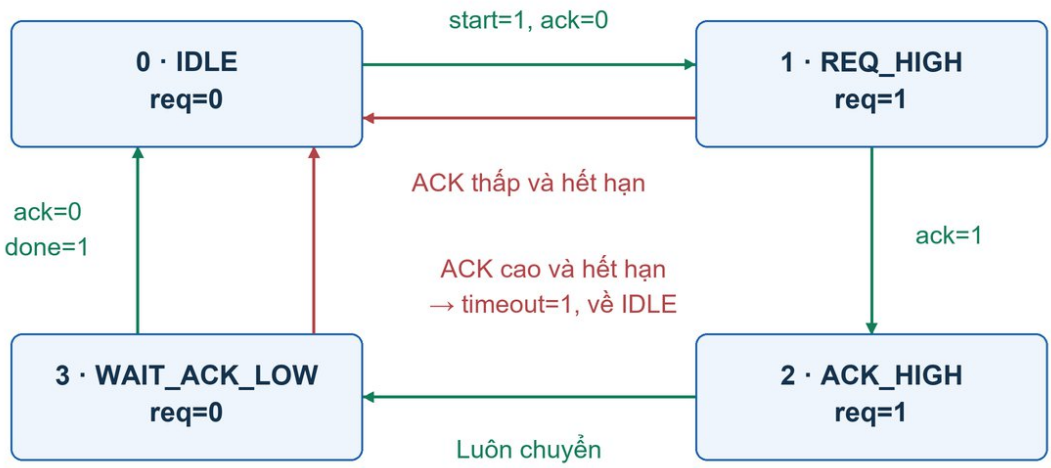
FSM có bốn trạng thái: IDLE (0), REQ_HIGH (1), ACK_HIGH (2), WAIT_ACK_LOW (3). REQ=1 ở trạng thái 1 hoặc 2. ACK_HIGH giữ REQ thêm một chu kỳ rồi chuyển sang chờ ACK hạ. DONE/TIMEOUT được mặc định xóa ở mỗi cạnh lên và chỉ bật ở nhánh kết thúc. [S2]



Hình 2a. Cấu trúc FSM, thanh ghi ngưỡng và bộ đếm chờ.

F10 · Sơ đồ chuyển trạng thái

Chuyển trạng thái tại cạnh lên clk; req là ngõ ra Moore.



Không có điều kiện chuyển: giữ state (trừ ACK_HIGH luôn chuyển).
Reset đồng bộ ưu tiên cao nhất; ACK hợp lệ thắng timeout tại deadline.
Nhánh default: trở về IDLE; timeout không bảo đảm đối tác đã reset.
Nguồn: RTL lõi · SHA-256: 10cca64461908d5c…

Hình 2b. Sơ đồ chuyển trạng thái của bộ điều khiển bắt tay.

Bảng 2.1 — Các quyết định thiết kế

#	Quyết định	Phương án khác đã cân nhắc	Lý do chọn
1	Giữ bốn mã state 4 bit của nguồn	Mã nhị phân 2 bit hoặc one-hot	Giữ lõi và cổng state quan sát; không đổi mã chỉ để giảm số FF.
2	Chốt N; xóa count trước mỗi pha chờ	Đọc N trực tiếp hoặc dùng chung một ngân sách	Giữ deadline ổn định khi input đổi; ngân sách riêng cho hai pha.
3	ACK đúng hạn thắng timeout; reset ưu tiên	Kiểm hết hạn trước ACK	Cho phản hồi hợp lệ tại deadline; tránh báo timeout sai.
4	ena chỉ chặn START mới	Dừng toàn FSM khi ena=0	Giao dịch đã nhận tiếp tục; không làm treo REQ giữa giao thức.

Đây là giải thích cấu trúc của bản RTL đang dùng, không phải bản ghi quyết định trước khi viết mã. Phần cốt lõi là thứ tự if (ack) trước nhánh timeout ở REQ_HIGH, và if (!ack) trước nhánh timeout ở WAIT_ACK_LOW. Mã đầy đủ ở Phụ lục A.

3. Kiểm chứng chức năng

Model đếm số chu kỳ còn lại độc lập với state nội bộ DUT. Stimulus đổi tại cạnh xuống; model lấy mẫu cạnh lên; checker đọc sau 1 ns để chờ nonblocking. Clock 10 ns, độ chính xác 1 ps. Bảng dưới đối chiếu mong đợi theo contract với CSV của lần chạy đã lưu. [E1]

Bảng 3.1 — Ca kiểm thử và kết quả

#	Ca kiểm thử	Đầu vào	Mong đợi	Thu được	Đạt?	Vì sao chọn ca này
1	TC01_NORMAL	N=8; ACK lên 50, hạ 70 ns	DONE 75–85 ns; timeout=0	Đúng; 65 check	PASS	Đủ bốn pha, đo latency
2	TC02_DELAYED_SUCCESS	N=8; ACK lên 130, hạ 170 ns	DONE 175–185 ns	Đúng; 125 check	PASS	Hai pha phản hồi chậm
3	TC03_NO_ACK	N=4; ACK luôn 0	TIMEOUT 235–245 ns; DONE=0	Đúng; 77 check	PASS	Bắt lệch một cạnh chờ
4	TC05_BACK_TO_BACK	N=2 rồi 9; hai START	Hai DONE riêng; nhận lần 2 ở 375 ns	Đúng; 177 check	PASS	Nạp lại cấu hình, không chồng giao dịch
5	TC04_ACK_STUCK — bổ sung	N=4; ACK giữ cao sau REQ↓	TIMEOUT 315–325 ns; DONE=0	Đúng; 101 check	PASS	Kiểm timeout pha trả về
6	SW_REQ_TIMEOUT_N000 — BIÊN	N=0; ACK=0	TIMEOUT 925–935 ns	Đúng; 38 check	PASS	N=0 không thành 256 chu kỳ

Lõi: 1.035 ca, 138.571 chu kỳ kiểm, 1.665.218 phép so sánh, 0 lỗi; wrapper: 11 ca, 708 check, 0 lỗi. Bộ ca còn kiểm ACK cũ, START khi bận, đổi N khi bận, reset giữa cạnh, bốn nhóm × 256 N và stress 4.096 chu kỳ (seed F1002026). “Check” không đồng nghĩa kịch bản độc lập. [E1, E2]

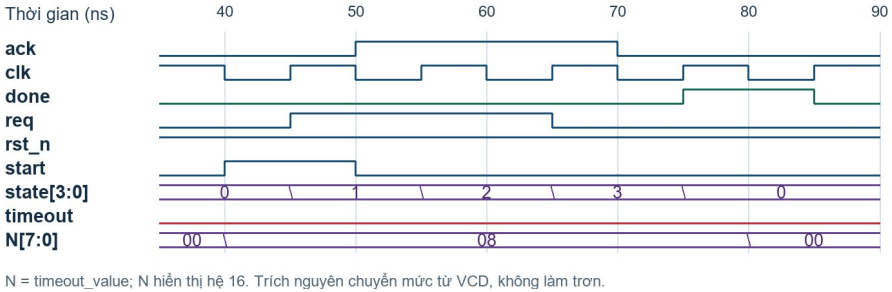
Bảng 3.2 — Phép thử phá mạch (bắt buộc)

Lỗi cố tình tạo ra	Số ca chuyển sang đỏ	Ca nào đỏ	Kết luận về chất lượng testbench
Deadline lệch một chu kỳ	2 / 8 ca được ghi trước khi dừng	TC03_NO_ACK; TC04_ACK_STUCK	Bắt sai thời điểm timeout ở cả hai pha chờ
Đảo ưu tiên ACK tại deadline	3 / 15 ca được ghi trước khi dừng	SW_LOW_TIMEOUT_N000; SW_REQ_PRIORITY_N000; SW_LOW_PRIORITY_N000	Phân biệt xử lý N=0 và ưu tiên ở deadline
Nạp lại N khi đang bận	1 / 8 ca được ghi trước khi dừng	TC07_BUSY_START	Bắt cấu hình thay đổi giữa giao dịch

Các tỷ lệ ở Bảng 3.2 đếm từ case_results.csv của từng mutant, không suy thành tỷ lệ trên 1.035 ca gốc. Ba mutant đều compile=0, simulation=1; FAIL/FATAL có chủ ý được lưu riêng. Không sửa RTL triển khai bằng các lỗi này. [E3]

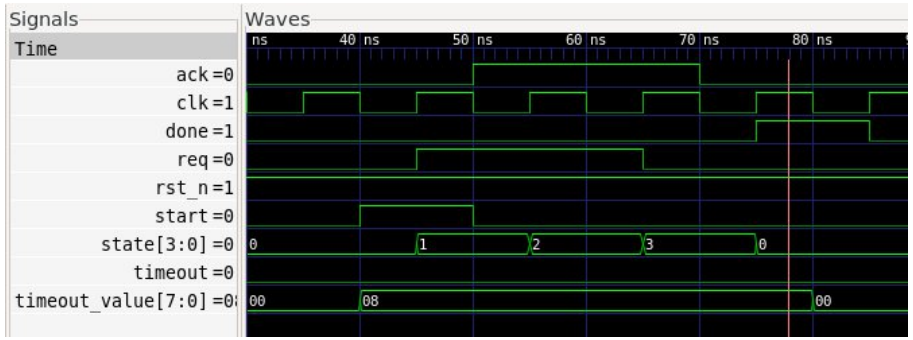
3.1. Bắt tay bình thường và đối tác đáp ứng chậm

Bắt tay bình thường (TC01_NORMAL)



N = timeout_value; N hiển thị hệ 16. Trích nguyên chuyển mức từ VCD, không làm trơn.

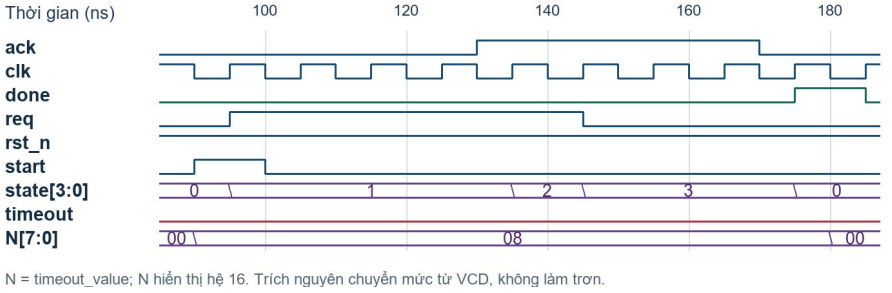
Hình 3a. Đồ thị từ VCD



Hình 3b. Ảnh GTKWave

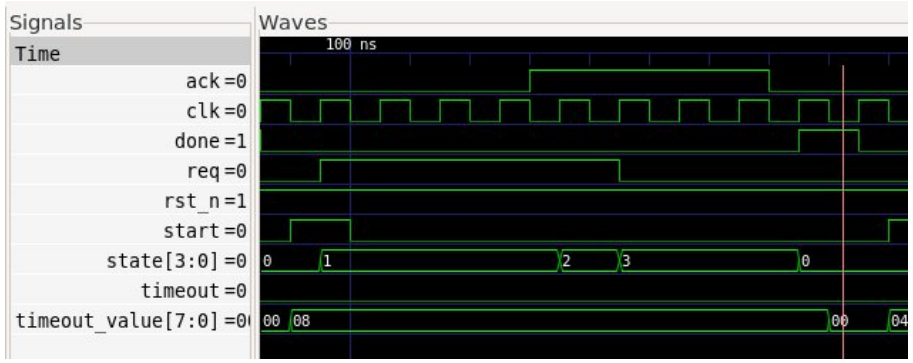
Bốn pha: REQ↑45 → ACK↑50 → REQ↓65 → ACK↓70 ns; DONE cao 75–85 ns; TIMEOUT=0. ACK được lấy mẫu tại 55 ns; state=2 giữ REQ thêm một chu kỳ. Latency REQ↑ đến DONE↑ là 30 ns. [E1]

Đối tác đáp ứng chậm (TC02_DELAYED_SUCCESS)



N = timeout_value; N hiển thị hệ 16. Trích nguyên chuyển mức từ VCD, không làm trơn.

Hình 4a. Đồ thị từ VCD

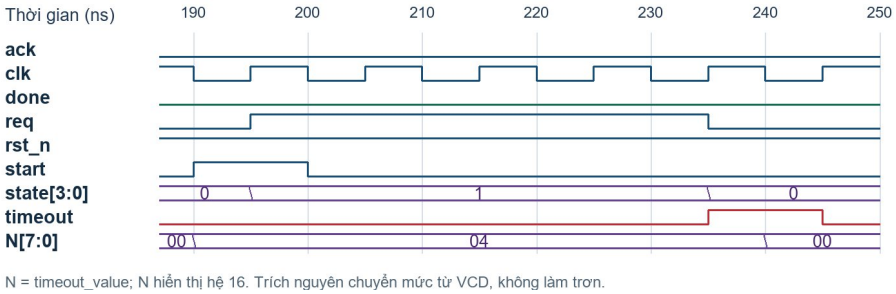


Hình 4b. Ảnh GTKWave

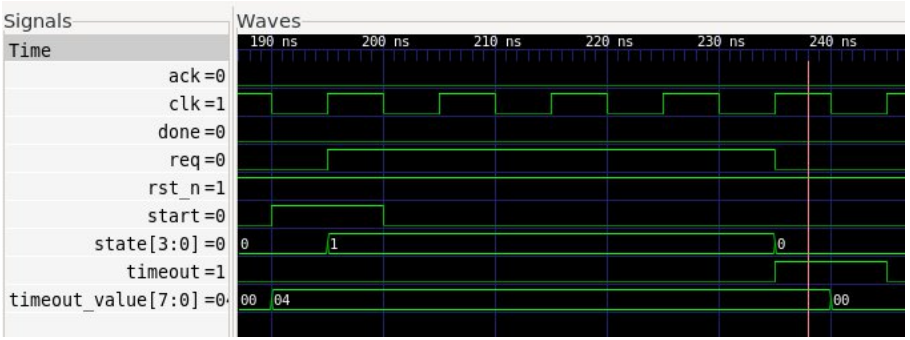
N=8; REQ↑95 → ACK↑130 → REQ↓145 → ACK↓170 ns; DONE cao 175–185 ns. ACK được lấy mẫu ở 135 ns; hai pha chờ đều trước hạn, TIMEOUT=0. [E1]

3.2. Timeout khi không có ACK và hai giao dịch liên tiếp

Không nhận được ACK (TC03_NO_ACK)



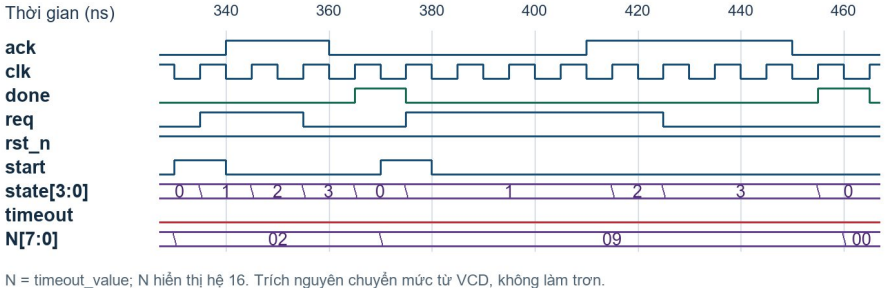
Hình 5a. Đồ thị từ VCD



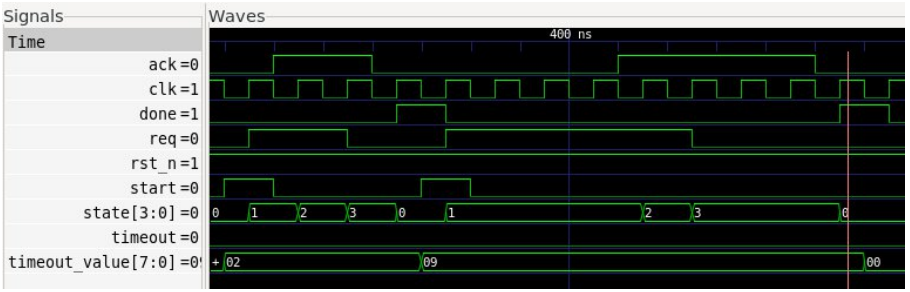
Hình 5b. Ảnh GTKWave

N=4; ACK=0; REQ cao 195–235 ns; TIMEOUT cao 235–245 ns; DONE=0. Bốn cạnh chờ là 205, 215, 225, 235 ns; không tính cạnh nhận START ở 195 ns. [E1]

Hai giao dịch liên tiếp (TC05_BACK_TO_BACK)



Hình 6a. Đồ thị từ VCD

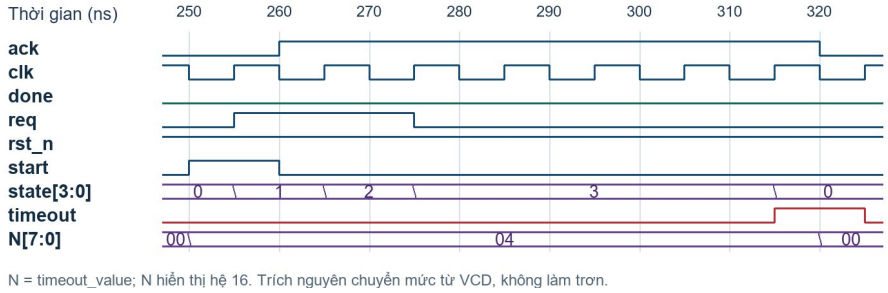


Hình 6b. Ảnh GTKWave

N=2 rồi N=9; DONE lần đầu cao 365–375 ns, REQ mới lên 375 ns, DONE lần hai cao 455–465 ns. START mới được nhận ở cạnh lên sau khi lần trước hoàn tất; không có giao dịch chồng nhau. [E1]

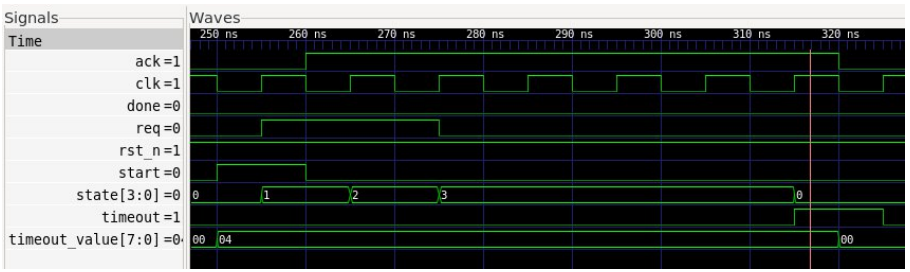
3.3. Timeout khi ACK giữ cao và giá trị biên N bằng 0

ACK giữ cao quá hạn (TC04_ACK_STUCK)



N = timeout_value; N hiển thị hệ 16. Trích nguyên chuyển mức từ VCD, không làm trơn.

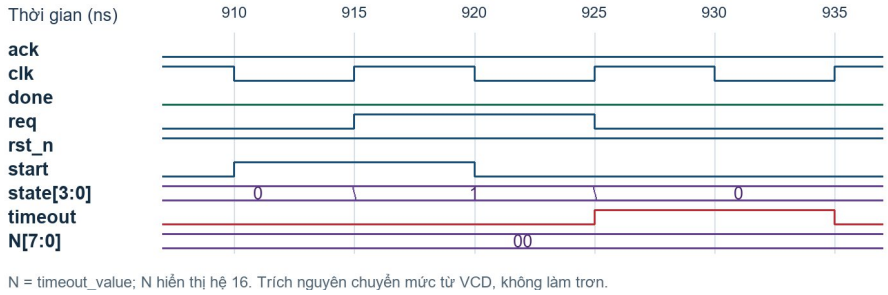
Hình 7a. Đồ thị từ VCD



Hình 7b. Ảnh GTKWave

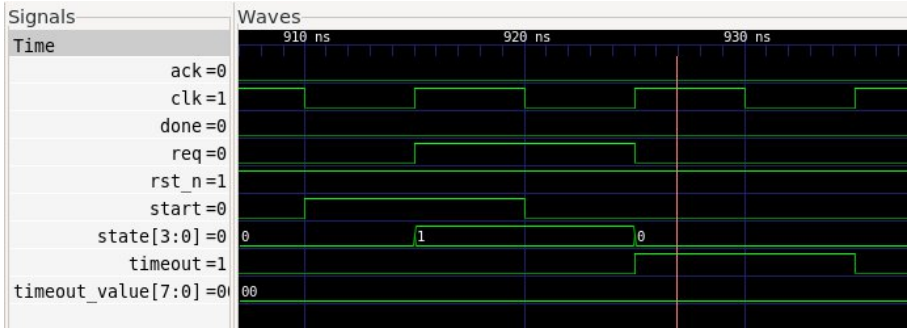
N=4; REQ cao 255–275 ns; ACK cao 260–320 ns. TIMEOUT cao 315–325 ns khi ACK còn cao, DONE=0. Ca bổ sung kiểm thời hạn ở pha chờ ACK hạ. [E1]

Giá trị biên N bằng 0 (SW_REQ_TIMEOUT_N000)



N = timeout_value; N hiển thị hệ 16. Trích nguyên chuyển mức từ VCD, không làm trơn.

Hình 8a. Đồ thị từ VCD



Hình 8b. Ảnh GTKWave

N=0; START lên 910 ns; REQ cao 915–925 ns; TIMEOUT cao 925–935 ns; ACK=0, DONE=0. Cạnh chờ đầu tiên kiểm ACK trước hết hạn; ACK hợp lệ đúng cạnh này vẫn được ưu tiên. [E1]

Các hình trích từ VCD đã lưu, không phải lần mô phỏng mới. Ảnh GTKWave được nhúng nguyên byte; số bên trái ảnh chụp là giá trị tại con trỏ, không phải giá trị cố định cho cả cửa sổ.

4. Kết quả tổng hợp

Tách hai mức: generic local Yosys 0.52 / CI 0.33 cho lõi 209 cell, top 210 cell; mapping Sky130 từ job vật lý dùng Yosys 0.62. Không dùng số 106 cell trong mô tả gói nguồn làm kết quả hiện tại. Hồ sơ không có bản ước lượng được lưu trước khi chạy. [E4, E6]

Bảng 4.1 — Số liệu tổng hợp

Chỉ số	Ước lượng của em (trước khi chạy)	Kết quả thật	Chênh lệch và giải thích
Tổng số standard cell	Chưa có bản ghi trước chạy	Mapping 133 cell; sau layout 454 cell, không filler	Khác stage: layout thêm tap, CTS và sửa timing; không so trực tiếp với generic.
Số flip-flop	Chưa có bản ghi trước chạy	22 FF ở generic và Sky130	Đối chiếu hồi cứu: state 4 + count 8 + giữ N 8 + DONE 1 + TIMEOUT 1 =22.
Diện tích (µm²)	Chưa có bản ghi trước chạy	Mapping 1.383,8272; final stdcell 2.162,07	Không dùng die 25.600 µm² hay filler làm diện tích logic.
Hệ số sử dụng (%)	—	10,6997% (final stdcell/core)	Final 2.162,07 / core 20.206,9 ×100; không phải utilization mục tiêu ban đầu.

Bảng 4.2 — Phân loại cell

Loại cell	Số lượng	Vai trò trong thiết kế của em
dfxtp_2	22	Thanh ghi state, count, N, DONE/TIMEOUT; tổng diện tích 467,9488 µm² (33,82%).
conb_1	19	Tie cells tạo mức hằng theo mapping thư viện.
xor2_2 + xnor2_2	3 + 7 =10	Mạng logic tổ hợp tổng hợp cho số học và so sánh.
buf_2 + inv_2	3 + 3 =6	Đệm/đảo trong netlist mapping trước CTS.
Các cổng còn lại và MUX	76	AND/OR/NAND/NOR/AOI/OAI và mux; cùng bốn nhóm trên thành 133 cell.

Trong thống kê mapping, dfxtp_2 là loại cell có tổng diện tích lớn nhất: 467,9488 µm²/1.383,8272 µm² ≈33,82%. Kết luận này lấy từ cột Local Area trong yosys-synthesis.log, không suy từ số lượng OR hay ANDNOT. [E6]

Generic giữ 4 bit trạng thái quan sát, không tự rút thành 2 FF dù bốn trạng thái chỉ cần 2 bit thông tin. Đối chiếu chân Q: count 8 và hai bit DONE/TIMEOUT dùng SDFF, timeout 8 và state 4 dùng SDFFE; tổng 22, không có latch. START_INTERNAL chưa dùng vẫn có cảnh báo lint. [E4]

5. Kết quả phân tích thời gian

STA dùng clock 10 ns, input/output delay 2 ns, uncertainty 0,25 ns, transition 0,15 ns; phân tích 9 góc RC/PVT. Đây là fallback SDC của flow, chưa phải hợp đồng giao tiếp bên ngoài đã được xác nhận. Mô phỏng clock 10 ns không tự chứng minh tốc độ phần cứng. [E6]

Bảng 5.1 — Số liệu thời gian

Chỉ số	Giá trị	Đạt / Vi phạm	Nhận xét
Chu kỳ ràng buộc (ns)	10,000	—	Mục tiêu 100 MHz, theo SDC đã lưu.
Worst setup slack (ns)	+1,314872	Đạt	max_ss_100C_1v60; WNS/TNS=0/0, không gọi WS dương là WNS.
Worst hold slack (ns)	+0,109147	Đạt	min_ff_n40C_1v95; WNS/TNS=0/0.
Tần số tối đa ước tính (MHz)	≈115,14	—	1000/(10−1,314871804); xấp xỉ STA, không phải Fmax đo thực.
Max transition / slew (ns)	0,933907 >0,750000	Vi phạm	11 chân cùng nhánh reset net20 tại mỗi góc SS.

Bảng 5.2 — Đường tới hạn

Hạng mục	Nội dung
Điểm xuất phát	_187_/Q: impl.dut.timeout_q[3], FF dfxtp_2.
Điểm kết thúc	_197_/D; FF lưu impl.dut.count_q[2].
Đi qua các khối	or4_2 → or4_2 → and3_2 → or2_2 → o311a_2 → a21oi_2 → a311oi_2 → hold44 (dlygate4sd3_1).
Số tầng logic	7 tầng tổ hợp + 1 delay cell trên data path; không tính FF và cây clock.
Vì sao chính đường này chậm nhất	Logic liên quan ngưỡng timeout và cập nhật counter; hai or4 đầu và delay cell hold44 đóng góp trễ đáng kể tại max-SS.
Một cách cụ thể để rút ngắn nó	Thử cân bằng logic điều kiện và rà hold44/drive; phải chạy lại đủ góc để tránh đổi setup lấy lỗi hold. Đây là đề xuất chưa chạy.

Arrival 8,831797 ns, required 10,146668 ns; chênh lệch 1,314872 ns. Clock rise skew: TT 43,615 ps; max-SS 75,705 ps. CTS có 22 sink, 5 buffer cây +2 dummy load, depth 2; raw skew khác effective skew có uncertainty/derate. [E6]

Giới hạn: còn 11 chân vi phạm slew trên net20; giảm tần số không tự làm sườn tín hiệu nhanh hơn. Cần xem tải/driver reset và thử chia tải hoặc đổi buffer, rồi kiểm lại STA/DRC/LVS. Không tuyên bố đạt mọi ràng buộc từ setup/hold dương.

6. Công suất và kiểm tra vật lý

Bảng 6.1 dùng corner nom_tt_025C_1v80 sau routing. Report gốc ghi Watts; số được đổi sang mW hoặc nW. Đây là ước tính vectorless, chưa nạp hoạt động workload VCD/SAIF; numeric toggle rate không được lưu. [E6]

Bảng 6.1 — Công suất và kiểm tra vật lý

Hạng mục	Giá trị / Kết quả	Nhận xét và giới hạn
Công suất động	0,285386559 mW	Internal + switching; phụ thuộc hoạt động chuyển mức.
Công suất tĩnh (rò)	15,852628 nW	Leakage theo corner/thư viện; không phải công suất đo silicon.
Tổng công suất	0,285402435 mW	Ước lượng; phụ thuộc giả định tần suất chuyển mức của công cụ.
DRC	Magic 0; KLayout 0; route 0 lỗi	Kiểm hình học theo deck; KHÔNG kiểm chức năng. Slew là kiểm khác, vẫn còn 11 chân vi phạm.
LVS	0 lỗi; Circuits match uniquely	Đối chiếu kết nối layout với netlist tham chiếu; không chứng minh đặc tả hoặc mọi chuỗi input.

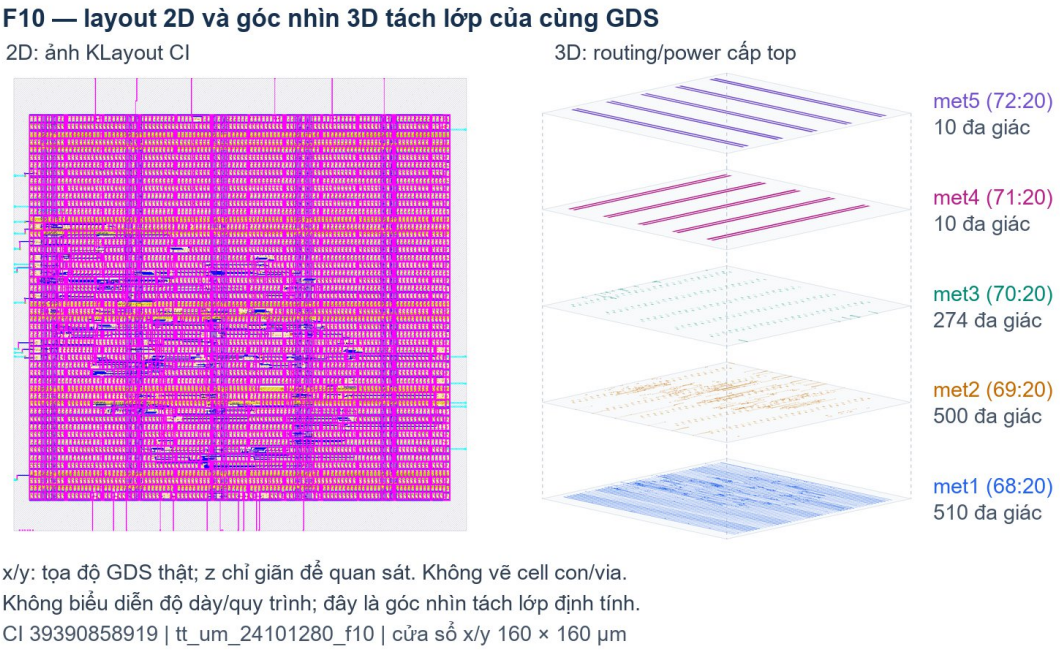
Công suất động chi phối ở TT: internal≈0,212633 mW và switching≈0,072754 mW; leakage chỉ≈0,000015853 mW. Clock group≈0,140511 mW, khoảng 49,2% tổng power trong report. Những tỷ lệ này vẫn phụ thuộc activity mặc định. [E6]

Final metric power__total=0,333318865 mW thuộc max_ff_n40C_1v95, không phải số TT trong bảng. Không trộn hai corner để kết luận mạch tiêu thụ công suất khác nhau do sửa RTL. [E6]

DRC sạch không có nghĩa mạch chạy đúng chức năng. DRC xét quy tắc hình học; LVS xét kết nối so với netlist; mô phỏng xét hành vi theo kích thích. Ba tầng bằng chứng không thay thế nhau.

Giới hạn flow: thiếu VSRC_LOC_FILES nên IR-drop dùng giả định vị trí nguồn; chưa thử nguồn/package thật, EM đầy đủ, gate-level/SDF hoặc phần cứng. GDS xuất được và LVS sạch không đủ để khẳng định sign-off. [E6]

7. Quan sát layout



Hình 9. Layout 2D từ KLayout CI và góc nhìn tách lớp định tính từ cùng GDS.

Bảng 7.1 — Quan sát layout

Nhìn thấy gì	Ở đâu trên hình	Liên hệ với bước quy trình nào
Hàng standard cell	Hình 9 bên trái, vùng core	Liên hệ khái niệm hình thành transistor và hàng cell; không suy transistor riêng từ ảnh tổng quan.
Các lớp kim loại	Hình 9 bên phải; met1…met5	Liên hệ các bước tạo cách điện, kim loại và via tuần tự; ảnh tách lớp không thể hiện đầy đủ via/hierarchy.
Vùng core và die	Die 160×160 µm; core(5,52;10,88)–(154,10;146,88) µm	Placement/routing nằm trong core; die area không bằng tổng diện tích logic.
Phân phối clock	CTS xác nhận 22 sink, 7 clock buffer, depth 2	Có cây clock vì thiết kế tuần tự; ảnh tổng quan chưa gắn nhãn từng nhánh để nhận dạng độc lập.

Góc nhìn tách lớp dùng 1.304 polygon kim loại cấp top, x/y từ GDS; z giãn tùy ý để quan sát. Không bung cell con, không dựng via hay bề dày công nghệ. Đây là nhận xét quan sát, không phải kết luận công nghệ chế tạo. [E6]

8. Trả lời câu phân tích riêng của đề

“Bắt tay bốn pha là nền của mọi giao tiếp giữa hai khối. Vẽ giản đồ thời gian bốn pha từ waveform mô phỏng của chính mình và chú thích từng pha; đây là phần trọng tâm của báo cáo đề này.” [S1, §4.2]

Bảng 8.1 — Số liệu dùng để trả lời

Số liệu	Giá trị	Lấy từ báo cáo nào
Pha 1 — phát yêu cầu	REQ↑45 ns; state 0→1	E1: VCD local, Hình 3a–3b
Pha 2 — đối tác xác nhận	ACK↑50 ns; được lấy mẫu 55 ns	E1: cùng VCD; TB đổi ở cạnh xuống
Pha 3 — requester hạ yêu cầu	REQ↓65 ns; state 2→3	E1: cùng VCD
Pha 4 — đối tác hạ xác nhận	ACK↓70 ns; được lấy mẫu 75 ns	E1: cùng VCD
Hoàn tất	DONE 75–85 ns; timeout=0	E1: TC01_NORMAL, CSV 37–87 ns
Latency của ca bình thường	REQ↑ đến DONE↑ =30 ns =3T	Hiệu hai mốc từ VCD; không phải trễ cổng
Timer và cấu hình	count_q 8 bit; timeout_q 8 bit; N từ ui_in[7:0]	RTL Phụ lục A; netlist generic E4
Giới hạn giao dịch hợp lệ	≤2×max(1,N)+1 chu kỳ sau nhận START	E5: N 0/N 1/N 255 cho 3/3/511 chu kỳ

Đọc Hình 3 phải phân biệt ACK lên 50 ns với cạnh FSM nhận ACK 55 ns. Trạng thái ACK_HIGH giữ REQ một chu kỳ; chỉ khi lấy mẫu ACK=0 trong WAIT_ACK_LOW mới phát DONE. Đây là bốn cạnh của tín hiệu giao thức, không phải bốn bit trạng thái.

Với N>0, count_q so với N−1: N=4 xét count=0,1,2,3 ở bốn cạnh chờ. N=0 có nhánh riêng ở cạnh chờ đầu tiên, nhưng phản hồi đúng hạn vẫn được ưu tiên. Bỏ timeout sẽ làm requester treo ở REQ_HIGH nếu ACK không lên, hoặc WAIT_ACK_LOW nếu ACK không hạ.

Sweep chỉ phủ bốn kịch bản trên mọi N, không phủ mọi chuỗi input. Chẩn đoán ngoài contract có ACK hạ 5280 ns trước REQ hạ 5285 ns mà DONE vẫn lên 5295 ns; không dùng quan sát này làm bằng chứng peer đúng bốn pha. Chưa có formal proof hoặc kiểm metastability. [E5]

9. Kết luận, giới hạn và tự đánh giá

Bảng 9.1 — Kết luận có điều kiện

Kết luận	Điều kiện đã tạo ra nó	Điều này KHÔNG chứng minh
RTL khớp model trên các ca đã chạy	Icarus 12; core 1.035 ca/0 lỗi; wrapper 11 ca/0 lỗi; seed F1002026	Không chứng minh mọi input, peer tuân thủ giao thức hoặc silicon hoạt động.
Tổng hợp giữ 22 FF, không có latch	Generic Yosys 0.52 local/0.33 CI; Sky130 mapping 0.62	Không chứng minh tối ưu diện tích hoặc cần đúng 4 FF trạng thái về mặt thông tin.
Có GDS, setup/hold và DRC/LVS theo artifacts	LibreLane 3.0.16; sky130A; T=10 ns; final utilization 10,6997%	Không đủ mọi ràng buộc: còn 11 chân slewSS; power vectorless và nguồn IR giả định.

Em tự đánh giá báo cáo chủ yếu ở mức Khá. Nội dung chính và bằng chứng đã có, nhưng một số chỗ diễn giải còn dài, chú thích layout chưa chi tiết và phần ước lượng trước khi chạy chưa có bản ghi.

Bảng 9.2 — Tự đánh giá theo rubric

Tiêu chí	Em tự đánh giá	Lý do
1. Đặc tả và sơ đồ khối	Khá	Em đã mô tả giao diện và FSM; chân START và phạm vi requester còn cần xác nhận.
2. RTL và kiểm chứng chức năng	Khá	Em đã trình bày ca bắt buộc, ca biên và 3 phép thử phá mạch; độ bao phủ vẫn có giới hạn.
3. Đọc báo cáo tổng hợp	Đạt	Em đã đối chiếu 22 FF và diện tích, nhưng chưa có bản ghi ước lượng trước khi chạy.
4. Đọc báo cáo thời gian	Khá	Em đã phân biệt setup/hold và truy đường tới hạn; còn 11 chân vi phạm slew ở góc SS.
5. Đọc công suất và vật lý	Khá	Em đã tách công suất động/rò và nêu giới hạn DRC/LVS; công suất vẫn là ước tính vectorless.
6. Quan sát layout	Khá	Em đã có ảnh layout; chú thích vị trí chân, hướng dây và nhánh clock còn chưa chi tiết.
7. Câu phân tích riêng	Khá	Em đã giải thích bốn pha bằng mốc VCD; phần phân biệt cạnh tín hiệu và cạnh lấy mẫu còn dài.
8. Kết luận có điều kiện	Khá	Em đã nêu điều kiện và giới hạn; một số nhận xét còn lặp, cần viết gọn hơn.
9. Trình bày và tái lập	Đạt	Em đã ghi repo, commit và công cụ; lịch sử hai commit chưa thể hiện rõ quá trình thực hiện.

Hồ sơ có dữ liệu chạy công cụ và các giới hạn đã ghi; không đồng nghĩa giảng viên đã nghiệm thu, đã chế tạo chip hoặc đã chứng minh hiểu bài. Phần ước lượng trước chạy không thể được thay bằng số hồi cứu. Không có khảo sát ba cấu hình physical độc lập để nhận phần điểm khảo sát tham số.

Phụ lục A — Mã nguồn RTL đầy đủ

Dự án dùng ba tệp SystemVerilog thay cho một src/project.v duy nhất trong ví dụ mẫu. Ba tệp dưới đây tạo đủ lõi, wrapper và top đã dùng; không dán netlist tổng hợp thay cho RTL.

A.1. Lõi bộ điều khiển bắt tay

Tệp: rtl/f10_handshake_controller.sv · SHA-256: 10cca64461908d5cbe1da1d36be07633c613b92f1c2daeff7fd2ba414cccdfe4

```
module f10_handshake_controller #(
  parameter int START_INTERNAL = 0
) (
  input  logic      clk,
  input  logic      rst_n,
  input  logic      start,
  input  logic      ack,
  input  logic [7:0] timeout_value,
  output logic      req,
  output logic      done,
  output logic      timeout,
  output logic [3:0] state
);

  typedef enum logic [3:0] {
    S_IDLE       = 4'b0000,
    S_REQ_HIGH   = 4'b0001,
    S_ACK_HIGH   = 4'b0010,
    S_WAIT_ACK_LOW = 4'b0011
  } state_t;

  state_t state_q;
  logic [7:0] timeout_q;
  logic [7:0] count_q;

  assign state = state_q;
  assign req   = (state_q == S_REQ_HIGH) || (state_q == S_ACK_HIGH);

  always_ff @(posedge clk) begin
    if (!rst_n) begin
      state_q   <= S_IDLE;
      timeout_q <= 8'd0;
      count_q   <= 8'd0;
      done      <= 1'b0;
      timeout   <= 1'b0;
    end else begin
      done    <= 1'b0;
      timeout <= 1'b0;

      case (state_q)
        S_IDLE: begin
          count_q <= 8'd0;
          if (start) begin
            timeout_q <= timeout_value;
```

```
            count_q   <= 8'd0;
            if (!ack)
              state_q <= S_REQ_HIGH;
            else
              state_q <= S_IDLE;
          end
        end

        S_REQ_HIGH: begin
          if (ack) begin
            count_q <= 8'd0;
            state_q <= S_ACK_HIGH;
          end else if (timeout_q == 8'd0) begin
            timeout <= 1'b1;
            state_q <= S_IDLE;
          end else if (count_q == timeout_q - 8'd1) begin
            timeout <= 1'b1;
            state_q <= S_IDLE;
          end else begin
            count_q <= count_q + 8'd1;
          end
        end

        S_ACK_HIGH: begin
          count_q <= 8'd0;
          state_q <= S_WAIT_ACK_LOW;
        end

        S_WAIT_ACK_LOW: begin
          if (!ack) begin
            done    <= 1'b1;
            state_q <= S_IDLE;
            count_q <= 8'd0;
          end else if (timeout_q == 8'd0) begin
            timeout <= 1'b1;
            state_q <= S_IDLE;
          end else if (count_q == timeout_q - 8'd1) begin
            timeout <= 1'b1;
            state_q <= S_IDLE;
          end else begin
            count_q <= count_q + 8'd1;
          end
        end

        default: begin
          state_q <= S_IDLE;
          count_q <= 8'd0;
          done    <= 1'b0;
          timeout <= 1'b0;
        end
      endcase
    end
  end
```

```
endmodule
```

A.2. Wrapper giao diện Tiny Tapeout

Tệp: rtl/f10_tiny_tapeout_top.sv · SHA-256:
4d08fe40a7a97fb2b4d9f5236fd8a1ceca01c6e529fb48b6502c569c7b09aa38

```
`default_nettype none

// Working integration assumption: uio_in[1] is START (option 2 in the
// supplied functional_spec). The charter does not explicitly assign START.
// ena suppresses NEW starts only; an accepted handshake must keep progressing.
module f10_tiny_tapeout_top (
   input  wire [7:0] ui_in,
   output wire [7:0] uo_out,
   input  wire [7:0] uio_in,
   output wire [7:0] uio_out,
   output wire [7:0] uio_oe,
   input  wire       ena,
   input  wire       clk,
   input  wire       rst_n
);
   wire req, done, timeout;
   wire [3:0] state;

   f10_handshake_controller dut (
      .clk(clk),
      .rst_n(rst_n),
      .start(ena && uio_in[1]),
      .ack(uio_in[0]),
      .timeout_value(ui_in),
      .req(req),
      .done(done),
      .timeout(timeout),
      .state(state)
   );

   assign uo_out  = {1'b0, state, timeout, done, req};
   assign uio_out = 8'b0;
   assign uio_oe  = 8'b0;
   wire _unused = &{uio_in[7:2], 1'b0};
endmodule

`default_nettype wire
```

A.3. Tên top của bài nộp

Tệp: rtl/tt_um_24101280_f10.sv · SHA-256:
30cfe7059d9e7572ea64f71c638876d24dcdb2a35eafa5c1b32f8415e0b1c5bb

```
`default_nettype none

// Tiny Tapeout top with the student's module name; no extra state or behavior.
module tt_um_24101280_f10 (
   input  wire [7:0] ui_in,
   output wire [7:0] uo_out,
   input  wire [7:0] uio_in,
   output wire [7:0] uio_out,
   output wire [7:0] uio_oe,
   input  wire       ena,
   input  wire       clk,
   input  wire       rst_n
);
   f10_tiny_tapeout_top impl (
      .ui_in(ui_in), .uo_out(uo_out), .uio_in(uio_in),
      .uio_out(uio_out), .uio_oe(uio_oe),
      .ena(ena), .clk(clk), .rst_n(rst_n)
   );
endmodule

`default_nettype wire
```

Phụ lục B — Testbench đầy đủ

Hai testbench chính kiểm lõi và wrapper; testbench chẩn đoán bổ sung kiểm giới hạn N và đối tác ngoài contract. Mã dưới đây là toàn bộ nội dung tệp, không rút gọn thành mô tả hoặc ảnh.

B.1. Testbench tự kiểm của lõi

Tệp: tb/f10_tb_selfcheck.sv · SHA-256: 03bb683e5cd412756abcb5a00318bb7422a4a1d9115e0d9c52aa5cfc907080a4

```
`timescale 1ns/1ps

// Independent, cycle-by-cycle checker for the supplied F10 core contract.
// Drive at falling edges; sample one nanosecond after rising edges, after NBA.
// The model uses a remaining-cycle deadline, never DUT state/internal registers.
module f10_tb_selfcheck;
  logic clk = 0;
  logic rst_n = 0, start = 0, ack = 0;
  logic [7:0] timeout_value = 0;
  wire req, done, timeout;
  wire [3:0] state;

  f10_handshake_controller dut (
    .clk(clk), .rst_n(rst_n), .start(start), .ack(ack),
    .timeout_value(timeout_value), .req(req), .done(done),
    .timeout(timeout), .state(state)
  );
  always #5 clk = ~clk;

  integer model_phase = 0;
  integer model_budget = 1;
  integer remaining = 0;
  logic model_req = 0, model_done = 0, model_timeout = 0;
  integer previous_phase = 0;
  logic sampled_ack = 0;
  logic previous_done = 0, previous_timeout = 0;
  integer busy_age = 0;

  integer cycles = 0, checks = 0, errors = 0;
  integer cases = 0, failed_cases = 0;
  integer case_start_cycle = 0, case_start_checks = 0, case_start_errors = 0;
  integer csv;
  string case_id = "BOOT";
  string case_description = "initial reset";
  time case_start_time;
  integer n, k, phase;
  integer random_resets = 0;
  logic [31:0] rng = 32'hF1002026;

  task automatic check(input logic condition, input string message);
    begin
      checks = checks + 1;
      if (condition !== 1'b1) begin
        errors = errors + 1;
        $display("FAIL case=%s cycle=%0d time=%0t: %s", case_id, cycles, $time, message);
```

```
            $display("  inputs rst_n=%b start=%b ack=%b N=%0d; DUT state=%b req=%b done=%b
timeout=%b; expected phase=%0d req=%b done=%b timeout=%b remaining=%0d",
                rst_n, start, ack, timeout_value, state, req, done, timeout,
                model_phase, model_req, model_done, model_timeout, remaining);
            if (errors >= 20) $fatal(1, "Stopping after 20 failures");
          end
        end
      endtask

      always @(posedge clk) begin
        cycles = cycles + 1;
        previous_phase = model_phase;
        sampled_ack = ack;
        model_done = 0;
        model_timeout = 0;

        if (!rst_n) begin
          model_phase = 0;
          model_budget = 1;
          remaining = 0;
          busy_age = 0;
        end else begin
          case (model_phase)
            0: begin
              remaining = 0;
              if (start && !ack) begin
                model_budget = (timeout_value == 0) ? 1 : timeout_value;
                remaining = model_budget;
                model_phase = 1;
              end
            end
            1: begin
              if (ack) begin
                model_phase = 2;
                remaining = 0;
              end else if (remaining <= 1) begin
                model_timeout = 1;
                model_phase = 0;
                remaining = 0;
              end else remaining = remaining - 1;
            end
            2: begin
              model_phase = 3;
              remaining = model_budget;
            end
            3: begin
              if (!ack) begin
                model_done = 1;
                model_phase = 0;
                remaining = 0;
              end else if (remaining <= 1) begin
                model_timeout = 1;
                model_phase = 0;
```

```
                remaining = 0;
            end else remaining = remaining - 1;
          end
          default: $fatal(1, "Reference model illegal phase");
        endcase
        if (model_phase == 0) busy_age = 0;
        else busy_age = busy_age + 1;
    end
    model_req = (model_phase == 1 || model_phase == 2);

    #1;
    check((^{state, req, done, timeout}) !== 1'bx, "Outputs must not contain X/Z after sampled reset");
    check(state === model_phase[3:0], "State differs from independent reference");
    check(req === model_req, "REQ differs from independent reference");
    check(done === model_done, "DONE differs from independent reference");
    check(timeout === model_timeout, "TIMEOUT differs from independent reference");
    check(state <= 4'd3, "State must be one of the four legal encodings");
    check(!(done && timeout), "DONE and TIMEOUT must be mutually exclusive");
    check(!(done && previous_done), "DONE pulse must last only one cycle");
    check(!(timeout && previous_timeout), "TIMEOUT pulse must last only one cycle");
    check(!(state == 0 && req), "REQ must be low in IDLE");
    check(!done || (rst_n && previous_phase == 3 && sampled_ack === 1'b0),
        "DONE requires sampled ACK-low in WAIT_ACK_LOW");
    check(busy_age <= 512, "Transaction must terminate within 512 sampled cycles");
    previous_done = done;
    previous_timeout = timeout;
end

task automatic cycle(input logic s, input logic a, input integer value, input logic reset_n);
    begin
        @(negedge clk);
        start = s;
        ack = a;
        timeout_value = value[7:0];
        rst_n = reset_n;
        @(posedge clk);
        #2; // Checker samples at +1 ns, then control returns at +2 ns.
    end
endtask

task automatic begin_case(input string id, input string description);
    begin
        case_id = id;
        case_description = description;
        case_start_cycle = cycles;
        case_start_checks = checks;
        case_start_errors = errors;
        case_start_time = $time;
    end
endtask

task automatic end_case;
    begin
```

```
        cases = cases + 1;
        if (errors != case_start_errors) failed_cases = failed_cases + 1;
        $fdisplay(csv, "%s,%s,%0d,%0d,%0d,%0d,%0d,%s", case_id, case_description,
            case_start_time, $time, cycles - case_start_cycle, checks - case_start_checks,
            errors - case_start_errors, (errors == case_start_errors) ? "PASS" : "FAIL");
        if (cases <= 10 || (errors != case_start_errors))
            $display("CASE %s %s time_ns=[%0d..%0d] cycles=%0d checks=%0d errors=%0d",
                case_id, (errors == case_start_errors) ? "PASS" : "FAIL",
                case_start_time, $time, cycles - case_start_cycle,
                checks - case_start_checks, errors - case_start_errors);
      end
    endtask

    task automatic idle_clear;
      begin
        cycle(0, 0, 0, 1);
        check({state, req, done, timeout} === 7'b0, "Idle recovery and result pulse clearing");
      end
    endtask

    task automatic complete_handshake(input integer value, input integer delay_up, input integer
delay_down);
      integer j;
      begin
        cycle(1, 0, value, 1);
        check(state === 4'd1 && req === 1'b1, "Phase 1 REQ-high");
        for (j = 0; j < delay_up; j = j + 1) cycle(0, 0, value, 1);
        cycle(0, 1, value, 1);
        check(state === 4'd2 && req === 1'b1, "Phase 2 ACK-high accepted while REQ stays high");
        cycle(0, 1, value, 1);
        check(state === 4'd3 && req === 1'b0, "Phase 3 REQ-low");
        for (j = 0; j < delay_down; j = j + 1) cycle(0, 1, value, 1);
        cycle(0, 0, value, 1);
        check(done === 1'b1 && timeout === 1'b0, "Phase 4 ACK-low and DONE");
      end
    endtask

    task automatic synchronous_reset_between_edges;
      logic [6:0] before_reset;
      begin
        @(negedge clk);
        before_reset = {state, req, done, timeout};
        start = 0;
        rst_n = 0;
        #1;
        check({state, req, done, timeout} === before_reset,
            "Synchronous reset must not change outputs before a rising edge");
        @(posedge clk);
        #2;
        check({state, req, done, timeout} === 7'b0,
            "Synchronous reset clears outputs at the next rising edge");
        cycle(0, 0, 0, 1);
      end
```

```
endtask

initial begin
  $dumpfile("f10_core_selfcheck.vcd");
  $dumpvars(0, clk, rst_n, start, ack, timeout_value, req, done, timeout, state,
            model_phase, model_req, model_done, model_timeout, remaining);
  csv = $fopen("case_results.csv", "w");
  if (!csv) $fatal(1, "Cannot open case_results.csv");
  $fdisplay(csv, "case_id,description,start_ns,end_ns,cycles,checks,errors,result");

  begin_case("TC00_RESET", "Reset and idle hold");
  cycle(0, 0, 0, 0);
  cycle(0, 0, 0, 1);
  idle_clear();
  end_case();

  begin_case("TC01_NORMAL", "Normal complete four-phase handshake");
  complete_handshake(8, 0, 0);
  idle_clear();
  end_case();

  begin_case("TC02_DELAYED_SUCCESS", "Delayed ACK-high and ACK-low before deadline");
  complete_handshake(8, 3, 2);
  idle_clear();
  end_case();

  begin_case("TC03_NO_ACK", "No ACK-high timeout after four waiting cycles");
  cycle(1, 0, 4, 1);
  repeat (3) begin
    cycle(0, 0, 4, 1);
    check(timeout === 1'b0 && req === 1'b1, "No early request-phase timeout");
  end
  cycle(0, 0, 4, 1);
  check(timeout === 1'b1 && done === 1'b0, "Request-phase timeout on deadline");
  idle_clear();
  end_case();

  begin_case("TC04_ACK_STUCK", "ACK-high stuck timeout after four waiting cycles");
  cycle(1, 0, 4, 1);
  cycle(0, 1, 4, 1);
  cycle(0, 1, 4, 1);
  repeat (3) begin
    cycle(0, 1, 4, 1);
    check(timeout === 1'b0 && state === 4'd3, "No early return-phase timeout");
  end
  cycle(0, 1, 4, 1);
  check(timeout === 1'b1 && done === 1'b0, "Return-phase timeout on deadline");
  idle_clear();
  end_case();

  begin_case("TC05_BACK_TO_BACK", "Two complete transactions without an idle bubble");
  complete_handshake(2, 0, 0);
  complete_handshake(9, 3, 2);
```

```
    idle_clear();
    end_case();

    begin_case("TC06_STALE_ACK", "Reject start while stale ACK is high and do not queue it");
    repeat (3) begin
      cycle(1, 1, 3, 1);
      check(state === 4'd0 && req === 1'b0, "Start with stale ACK is rejected");
    end
    cycle(0, 0, 3, 1);
    check(state === 4'd0 && req === 1'b0, "Rejected start is not queued");
    complete_handshake(3, 0, 0);
    idle_clear();
    end_case();

    begin_case("TC07_BUSY_START", "Ignore start in all three busy states");
    cycle(1, 0, 5, 1);
    cycle(1, 0, 1, 1);
    cycle(1, 1, 255, 1);
    cycle(1, 1, 0, 1);
    cycle(1, 1, 1, 1);
    cycle(1, 0, 255, 1);
    check(done === 1'b1 && state === 4'd0, "Busy start must not interrupt completion");
    idle_clear();
    end_case();

    begin_case("TC08_LATCH_TIMEOUT", "Live timeout input changes do not reload either wait
deadline");
    cycle(1, 0, 4, 1);
    cycle(1, 0, 0, 1);
    cycle(0, 0, 255, 1);
    cycle(1, 0, 1, 1);
    cycle(0, 0, 255, 1);
    check(timeout === 1'b1, "Request deadline keeps accepted N=4");
    idle_clear();
    cycle(1, 0, 4, 1);
    cycle(1, 1, 1, 1);
    cycle(1, 1, 0, 1);
    cycle(0, 1, 255, 1);
    cycle(1, 1, 1, 1);
    cycle(0, 1, 0, 1);
    cycle(1, 1, 255, 1);
    check(timeout === 1'b1, "Return deadline keeps accepted N=4");
    idle_clear();
    end_case();

    begin_case("TC09_SYNC_RESET", "Reset between edges in IDLE and all three busy states");
    for (phase = 0; phase < 4; phase = phase + 1) begin
      if (phase > 0) cycle(1, 0, 8, 1);
      if (phase > 1) cycle(0, 1, 8, 1);
      if (phase > 2) cycle(0, 1, 8, 1);
      check(state === phase[3:0], "Reset target phase reached");
      synchronous_reset_between_edges();
    end
```

```
    end_case();

    // Every value of the 8-bit configuration domain, both waiting phases,
    // both deadline outcomes. N=0 means one first waiting-edge decision.
    for (n = 0; n < 256; n = n + 1) begin
      begin_case($sformatf("SW_REQ_TIMEOUT_N%03d", n), "Request wait timeout at exact
deadline");
      cycle(1, 0, n, 1);
      for (k = 1; k < ((n == 0) ? 1 : n); k = k + 1) cycle(0, 0, n, 1);
      cycle(0, 0, n, 1);
      check(timeout === 1'b1 && done === 1'b0, "Request sweep exact timeout");
      idle_clear();
      end_case();

      begin_case($sformatf("SW_LOW_TIMEOUT_N%03d", n), "Return wait timeout at exact deadline");
      cycle(1, 0, n, 1);
      cycle(0, 1, n, 1);
      cycle(0, 1, n, 1);
      for (k = 1; k < ((n == 0) ? 1 : n); k = k + 1) cycle(0, 1, n, 1);
      cycle(0, 1, n, 1);
      check(timeout === 1'b1 && done === 1'b0, "Return sweep exact timeout");
      idle_clear();
      end_case();

      begin_case($sformatf("SW_REQ_PRIORITY_N%03d", n), "ACK-high wins over timeout on exact
deadline");
      cycle(1, 0, n, 1);
      for (k = 1; k < ((n == 0) ? 1 : n); k = k + 1) cycle(0, 0, n, 1);
      cycle(0, 1, n, 1);
      check(state === 4'd2 && timeout === 1'b0, "ACK-high priority on request deadline");
      cycle(0, 1, n, 1);
      cycle(0, 0, n, 1);
      check(done === 1'b1 && timeout === 1'b0, "Priority transaction completes all four phases");
      idle_clear();
      end_case();

      begin_case($sformatf("SW_LOW_PRIORITY_N%03d", n), "ACK-low wins over timeout on exact
deadline");
      cycle(1, 0, n, 1);
      cycle(0, 1, n, 1);
      cycle(0, 1, n, 1);
      for (k = 1; k < ((n == 0) ? 1 : n); k = k + 1) cycle(0, 1, n, 1);
      cycle(0, 0, n, 1);
      check(done === 1'b1 && timeout === 1'b0, "ACK-low priority on return deadline");
      idle_clear();
      end_case();
    end

    begin_case("TC10_DETERMINISTIC_STRESS", "4096 synchronous input cycles seed F1002026 plus
bounded drain");
    for (k = 0; k < 4096; k = k + 1) begin
      rng = {rng[30:0], rng[31] ^ rng[21] ^ rng[1] ^ rng[0]};
      if (rng[15:8] == 8'h00) random_resets = random_resets + 1;
```

```
        cycle(rng[0], rng[1], rng[23:16], rng[15:8] != 8'h00);
      end
      for (k = 0; k < 513 && model_phase != 0; k = k + 1) cycle(0, 0, 0, 1);
      idle_clear();
      check(model_phase == 0, "Stress drain leaves no transaction outstanding");
      end_case();

      check(cases == 1035, "All directed and sweep cases must execute");
      check(checks > 0, "A pass requires nonzero checks");
      check(failed_cases == 0, "No case may fail");
      $fdisplay(csv, "SUMMARY,all cases,0,%0d,%0d,%0d,%0d,%s", $time, cycles, checks, errors,
        (errors == 0 && failed_cases == 0) ? "PASS" : "FAIL");
      $fclose(csv);
      $display("SUMMARY cases=%0d failed_cases=%0d cycles=%0d checks=%0d errors=%0d
seed=F1002026 stress_resets=%0d",
        cases, failed_cases, cycles, checks, errors, random_resets);
      if (errors != 0 || failed_cases != 0) $fatal(1, "F10 SELF-CHECK FAILED");
      $display("ALL TESTS PASSED: F10 core independent reference and deadline sweep");
      $finish;
    end

    initial begin
      #2000000;
      $fatal(1, "Global 2 ms simulation watchdog expired");
    end
endmodule
```

B.2. Testbench wrapper và ánh xạ chân

Tệp: tb/f10_wrapper_tb.sv · SHA-256: 1b02007b98304098460a44a6f6c189f8546c19d4cb3a506eb18d4badc5e3da55

```
`timescale 1ns/1ps
`default_nettype none

// Concrete expected outputs for the complete TT pin interface.
// Stimulus at negedge; checks after following posedge/NBA update.
module f10_wrapper_tb;
  logic clk = 0;
  logic rst_n = 0, ena = 0;
  logic [7:0] ui_in = 0, uio_in = 0;
  wire [7:0] uo_out, uio_out, uio_oe;
  integer checks = 0, errors = 0, cases = 0, failed_cases = 0;
  integer before_errors = 0, before_checks = 0;
  integer csv, k;
  string current_case = "BOOT";
  time begin_time;

  tt_um_24101280_f10 dut (
    .ui_in(ui_in), .uo_out(uo_out), .uio_in(uio_in),
    .uio_out(uio_out), .uio_oe(uio_oe),
    .ena(ena), .clk(clk), .rst_n(rst_n)
  );
  always #5 clk = ~clk;

  task automatic check(input logic condition, input string message);
    checks = checks + 1;
    if (condition !== 1'b1) begin
      errors = errors + 1;
      $display("FAIL case=%s time=%0t %s inputs:rst=%b ena=%b ui=%h uio=%h out=%h",
current_case, $time, message, rst_n, ena, ui_in, uio_in, uo_out);
    end
  endtask

  task automatic outputs(input logic [7:0] expected);
    check((^{uo_out, uio_out, uio_oe}) !== 1'bx, "all output bits known after reset");
    check(uo_out === expected, $sformatf("full uo_out expected=%h actual=%h", expected, uo_out));
    check(uio_out === 8'b0, "uio_out tied low");
    check(uio_oe === 8'b0, "all bidirectional pins are inputs");
  endtask

  task automatic step(
    input logic reset_n, enable, start, ack,
    input logic [7:0] deadline,
    input logic [5:0] unused_bits,
    input logic [3:0] expected_state,
    input logic expected_req, expected_done, expected_timeout
  );
    @(negedge clk);
    rst_n = reset_n;
    ena = enable;
    ui_in = deadline;
```

```
    uio_in = {unused_bits, start, ack};
    @(posedge clk);
    #1;
    outputs({1'b0, expected_state, expected_timeout, expected_done, expected_req});
  endtask

  task automatic begin_case(input string id, input string description);
    current_case = id;
    cases = cases + 1;
    before_errors = errors;
    before_checks = checks;
    begin_time = $time;
    $display("CASE_BEGIN %s %s", id, description);
  endtask

  task automatic end_case;
    if (errors != before_errors) failed_cases = failed_cases + 1;
    $display("CASE_END %s checks=%0d errors=%0d", current_case, checks-before_checks, errors-
before_errors);
    $fdisplay(csv, "%s,%0t,%0t,%0d,%0d", current_case, begin_time, $time, checks-before_checks,
errors-before_errors);
  endtask

  initial begin
    $dumpfile("f10_wrapper.vcd");
    $dumpvars(0, f10_wrapper_tb);
    csv = $fopen("wrapper_cases.csv", "w");
    if (!csv) $fatal(1, "cannot open wrapper case CSV");
    $fdisplay(csv, "case,start_ps,end_ps,checks,errors");

    begin_case("WR01", "synchronous initial reset and complete ports");
    step(0,1,1,1,8'hff,6'h3f,0,0,0,0);
    step(1,1,0,0,8'd4,0,0,0,0,0);
    end_case();

    begin_case("WR02", "ena zero suppresses new start");
    repeat (3) step(1,0,1,0,8'd4,6'h3f,0,0,0,0);
    end_case();

    begin_case("WR03", "normal four-phase handshake through mapped pins");
    step(1,1,1,0,8'd4,0,1,1,0,0);
    step(1,1,0,0,8'd4,0,1,1,0,0);
    step(1,1,0,1,8'd4,0,2,1,0,0);
    step(1,1,0,1,8'd4,0,3,0,0,0);
    step(1,1,0,0,8'd4,0,0,0,1,0);
    step(1,1,0,0,8'd4,0,0,0,0,0);
    end_case();

    begin_case("WR04", "no ACK expires after exactly three wait edges");
    step(1,1,1,0,8'd3,0,1,1,0,0);
    step(1,1,0,0,8'd3,0,1,1,0,0);
    step(1,1,0,0,8'd3,0,1,1,0,0);
    step(1,1,0,0,8'd3,0,0,0,0,1);
```

```
step(1,1,0,0,8'd3,0,0,0,0,0);
end_case();

begin_case("WR05", "stale ACK blocks start; fresh start completes normally");
repeat (2) step(1,1,1,1,8'd4,0,0,0,0,0);
step(1,1,0,0,8'd4,0,0,0,0,0);
step(1,1,1,0,8'd4,0,1,1,0,0);
step(1,1,0,1,8'd4,0,2,1,0,0);
step(1,1,0,1,8'd4,0,3,0,0,0);
step(1,1,0,0,8'd4,0,0,0,1,0);
step(1,1,0,0,8'd4,0,0,0,0,0);
end_case();

begin_case("WR06", "ena falling does not freeze a busy transaction");
step(1,1,1,0,8'd4,0,1,1,0,0);
step(1,0,1,0,8'd4,0,1,1,0,0);
step(1,0,1,1,8'd4,0,2,1,0,0);
step(1,0,1,1,8'd4,0,3,0,0,0);
step(1,0,1,0,8'd4,0,0,0,1,0);
step(1,0,1,0,8'd4,0,0,0,0,0);
end_case();

begin_case("WR07", "unused uio input bits do not change outputs");
step(1,1,0,0,8'd4,6'h3f,0,0,0,0);
step(1,1,0,0,8'd4,6'h15,0,0,0,0);
step(1,1,0,0,8'd4,6'h2a,0,0,0,0);
step(1,1,0,0,8'd4,6'h00,0,0,0,0);
end_case();

begin_case("WR08", "all eight timeout input bits preserved including bit seven");
step(1,1,1,0,8'h80,0,1,1,0,0);
for (k=0; k<127; k=k+1) step(1,1,0,0,8'h80,0,1,1,0,0);
step(1,1,0,0,8'h80,0,0,0,0,1);
step(1,1,0,0,8'h80,0,0,0,0,0);
end_case();

begin_case("WR09", "zero timeout becomes one wait edge");
step(1,1,1,0,8'd0,0,1,1,0,0);
step(1,1,0,0,8'd0,0,0,0,0,1);
step(1,1,0,0,8'd0,0,0,0,0,0);
end_case();

begin_case("WR10", "stuck high ACK expires in wait-for-low phase");
step(1,1,1,0,8'd2,0,1,1,0,0);
step(1,1,0,1,8'd2,0,2,1,0,0);
step(1,1,0,1,8'd2,0,3,0,0,0);
step(1,1,0,1,8'd2,0,3,0,0,0);
step(1,1,0,1,8'd2,0,0,0,0,1);
step(1,1,0,0,8'd2,0,0,0,0,0);
end_case();

begin_case("WR11", "reset asserted between clocks acts only at next posedge");
step(1,1,1,0,8'd4,0,1,1,0,0);
```

```
    #1 rst_n = 0;
    #1 outputs(8'h09); // state=1 and req=1: unchanged before next posedge.
    @(posedge clk); #1 outputs(8'h00);
    step(1,1,0,0,8'd4,0,0,0,0,0);
    end_case();

    $fclose(csv);
    $display("WRAPPER_SUMMARY cases=%0d failed_cases=%0d checks=%0d errors=%0d", cases,
failed_cases, checks, errors);
    if (errors != 0 || failed_cases != 0 || checks == 0) $fatal(1, "wrapper test failed");
    $display("WRAPPER_ALL_TESTS_PASSED");
    $finish;
  end

  initial begin
    #100000;
    $fatal(1, "wrapper test timeout");
  end
endmodule

`default_nettype wire
```

B.3. Testbench chẩn đoán giới hạn giao dịch

Tệp: diagnostics/contract_bounds/f10_contract_bounds_tb.sv · SHA-256: 0929b577ef966f533b77b7a5ad59ed8d9152de2e33d68d4d7cf32a5be2ebfefd

```
`timescale 1ns/1ps

// Supplemental diagnostics only: original 1035-case suite is unchanged.
// Drive at negedge, sample 1 ns after posedge/NBA. Legal ACK stays high
// until REQ has fallen. The last experiment deliberately violates that rule.
module f10_contract_bounds_tb;
  logic clk = 0;
  logic rst_n = 0, start = 0, ack = 0;
  logic [7:0] timeout_value = 0;
  wire req, done, timeout;
  wire [3:0] state;
  integer cycles = 0, checks = 0, errors = 0;
  integer valid_cases = 0, unsupported_observations = 0;
  integer csv;
  bit tracking = 0;
  string current_case = "RESET";
  time req_up_time, ack_up_time, req_down_time, ack_down_time;
  time sampled_edge_time;

  f10_handshake_controller dut (
    .clk(clk), .rst_n(rst_n), .start(start), .ack(ack),
    .timeout_value(timeout_value), .req(req), .done(done),
    .timeout(timeout), .state(state)
  );
  always #5 clk = ~clk;
  always @(posedge clk) cycles = cycles + 1;
  always @(posedge req) if (tracking) req_up_time = $time;
  always @(negedge req) if (tracking) req_down_time = $time;
```

```
  always @(posedge ack) if (tracking) ack_up_time = $time;
  always @(negedge ack) if (tracking) ack_down_time = $time;

  task automatic check(input logic condition, input string message);
    checks = checks + 1;
    if (condition !== 1'b1) begin
      errors = errors + 1;
      $display("FAIL case=%s time=%0t cycle=%0d %s state=%b req=%b done=%b timeout=%b
ack=%b",
        current_case, $time, cycles, message, state, req, done, timeout, ack);
    end
  endtask

  task automatic step(
    input logic s, a, input integer n,
    input logic [3:0] expected_state,
    input logic expected_req, expected_done, expected_timeout
  );
    @(negedge clk);
    start = s;
    ack = a;
    timeout_value = n[7:0];
    @(posedge clk);
    sampled_edge_time = $time;
    #1;
    check({state, req, done, timeout} ===
        {expected_state, expected_req, expected_done, expected_timeout},
        "concrete expected outputs at sampled edge");
    check((^{state, req, done, timeout}) !== 1'bx, "known outputs");
    check(!(done && timeout), "exclusive result events");
  endtask

  task automatic begin_tracking(input string id);
    current_case = id;
    req_up_time = 0;
    ack_up_time = 0;
    req_down_time = 0;
    ack_down_time = 0;
    tracking = 1;
  endtask

  task automatic valid_both_deadlines(input integer n);
    integer w, k, accepted_cycle, latency;
    time accepted_time, done_time;
    w = (n == 0) ? 1 : n;
    begin_tracking($sformatf("VALID_BOTH_DEADLINES_N%03d", n));
    step(1, 0, n, 1, 1, 0, 0);
    accepted_cycle = cycles;
    accepted_time = sampled_edge_time;
    for (k=1; k<w; k=k+1) step(0, 0, n, 1, 1, 0, 0);
    // ACK rises on the final permitted request-wait edge.
    step(0, 1, n, 2, 1, 0, 0);
    step(0, 1, n, 3, 0, 0, 0);
```

```
      // ACK stays high beyond REQ falling, then falls on the final
      // permitted return-wait edge. Both phase budgets are consumed.
      for (k=1; k<w; k=k+1) step(0, 1, n, 3, 0, 0, 0);
      step(0, 0, n, 0, 0, 1, 0);
      latency = cycles - accepted_cycle;
      done_time = sampled_edge_time;
      check(latency == 2*w+1, "accepted-start to DONE equals two budgets plus ACK_HIGH hold");
      check(req_up_time == accepted_time, "REQ rises on the accepted start edge");
      check(req_up_time < ack_up_time && ack_up_time < req_down_time &&
          req_down_time < ack_down_time && ack_down_time < done_time,
          "legal peer: four signal edges strictly ordered before DONE");
      check((done_time-accepted_time) == latency*10,
          "cycle latency agrees with raw edge timestamps at 10 ns clock");
      valid_cases = valid_cases + 1;
      $display("VALID_PEER_RESULT N=%0d W=%0d accepted_ns=%0d done_ns=%0d
latency_cycles=%0d edge_ns=[%0d,%0d,%0d,%0d]",

n,w,accepted_time,done_time,latency,req_up_time,ack_up_time,req_down_time,ack_down_time);
      $fdisplay(csv,"%s,VALID_PEER,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d",
        current_case,n,w,accepted_time,done_time,latency,
        req_up_time,ack_up_time,req_down_time,ack_down_time);
      tracking = 0;
      step(0, 0, n, 0, 0, 0, 0);
    endtask

    task automatic observe_unsupported_early_ack_drop;
      integer accepted_cycle, latency;
      time accepted_time, done_time;
      begin_tracking("EARLY_ACK_DROP");
      step(1, 0, 8, 1, 1, 0, 0);
      accepted_cycle = cycles;
      accepted_time = sampled_edge_time;
      step(0, 1, 8, 2, 1, 0, 0);
      check(req === 1'b1 && ack === 1'b1, "about to violate peer hold rule during ACK_HIGH");
      // Deliberate unsupported peer behavior: ACK falls at the next
      // negedge while REQ is still high, before core lowers REQ.
      step(0, 0, 8, 3, 0, 0, 0);
      check(ack_down_time < req_down_time, "experiment actually violated ACK hold ordering");
      step(0, 0, 8, 0, 0, 1, 0);
      latency = cycles - accepted_cycle;
      done_time = sampled_edge_time;
      check(latency == 3, "observed core completes despite unsupported early ACK drop");
      unsupported_observations = unsupported_observations + 1;
      $display("OBSERVED_UNSUPPORTED_PEER N=8 accepted_ns=%0d done_ns=%0d
latency_cycles=%0d edge_ns=[%0d,%0d,%0d,%0d] ACK_fell_before_REQ: not a valid-protocol PASS",
        accepted_time,done_time,latency,req_up_time,ack_up_time,req_down_time,ack_down_time);
      $fdisplay(csv,"%s,OBSERVED_UNSUPPORTED_PEER,8,8,%0d,%0d,%0d,%0d,%0d,%0d,%0d",
        current_case,accepted_time,done_time,latency,
        req_up_time,ack_up_time,req_down_time,ack_down_time);
      tracking = 0;
      step(0, 0, 8, 0, 0, 0, 0);
    endtask
```

```
  initial begin
    $dumpfile("f10_contract_bounds.vcd");
    $dumpvars(0, f10_contract_bounds_tb);
    csv = $fopen("diagnostic_cases.csv", "w");
    if (!csv) $fatal(1,"cannot open diagnostic CSV");

$fdisplay(csv,"case_id,classification,N,W,accepted_ns,result_ns,latency_cycles,req_up_ns,ack_up_ns,req
_down_ns,ack_down_ns");
    step(0, 0, 0, 0, 0, 0, 0);
    rst_n = 1;
    step(0, 0, 0, 0, 0, 0, 0);
    valid_both_deadlines(0);
    valid_both_deadlines(1);
    valid_both_deadlines(255);
    observe_unsupported_early_ack_drop();
    check(valid_cases == 3 && unsupported_observations == 1, "all four diagnostic experiments
executed");
    check(checks > 0, "diagnostic checker executed");
    $fclose(csv);
    $display("DIAGNOSTIC_SUMMARY valid_cases=%0d unsupported_observations=%0d checks=%0d
errors=%0d",
      valid_cases,unsupported_observations,checks,errors);
    if (errors != 0) $fatal(1,"diagnostic expectation mismatch");
    $display("DIAGNOSTIC_COMPLETED: valid-peer bounds measured; unsupported peer observation
is a documented limit");
    $finish;
  end

  initial begin
    #100000;
    $fatal(1,"100 us diagnostic watchdog expired");
  end
endmodule
```

Phụ lục C — Thông tin tái lập

Đường dẫn kho GitHub	https://github.com/lechihoangcuong/eee703128-f10-24101280
Mã commit dùng cho báo cáo	CI thực thi: b59c838b69c1b35e73c9c9ca8cdcd36c34d7d25f Mã nguồn nộp: 7ae2df482cbf7f316b2eac5e066c0b33ebad88eb
Số hiệu / đường dẫn lần chạy CI	Run 40179312464, attempt 1 (artifacts đã lưu). https://github.com/lechihoangcuong/eee703128-f10-24101280/actions/runs/40179312464
Phiên bản LibreLane	3.0.16 · revision a09d248c49bd1f7170d41885936f63e4255b1fe5
Phiên bản PDK	sky130A · Ciel 8afc8346a57fe1ab7934ba5a6056ea8b43078e71 · sky130_fd_sc_hd
Ràng buộc chu kỳ xung nhịp	10 ns; in/out delay 2 ns; uncertainty 0,25 ns; transition 0,15 ns; max transition 0,75 ns
Ngày chạy	25/10/2026 UTC · run 09:06:56–09:10:10 · physical RUN_2026-10-25_09-08-50
Công cụ AI đã dùng (nếu có) và dùng vào việc gì	ChatGPT/Codex: hỗ trợ phân tích tài liệu, đối chiếu kết quả, dựng hình.

Lệnh tái lập và nguồn bằng chứng

Từ thư mục gốc repo: bash scripts/run_sim.sh (lõi và wrapper); bash scripts/run_synth.sh (generic); bash scripts/run_mutations.sh; bash diagnostics/contract_bounds/run.sh. Giữ version/hash/CSV/log/VCD và exit code, không chỉ dòng PASS.

Công cụ: local/CI chức năng Icarus 12; generic localYosys 0.52 / CI 0.33. Job physical: Yosys 0.62, Verilator 5.044, OpenROAD package 2026-02-17, Magic 8.3.623, Netgen 1.5.316, KLayout 0.30.7. Phiên bản và revision chi tiết trong CI_EVIDENCE.md.

Nguồn [S1] charter F10 đầy đủ, §4.2–6,9–12; [S2] ZIP F10 đã cung cấp; [S3] hướng dẫn Canvas 30065 (file4677304,4677309); [S4] MAU_BAO_CAO_B2 trong gói Lab guide giảng viên. Không dùng số liệu của báo cáo mẫu đã điền cho đề khác.

Bằng chứng [E1] evidence/local-run/core: simulation.log, case_results.csv, f10_core_selfcheck.vcd; [E2] local-run/wrapper; [E3] portable-validation/mutation_smoke; [E4] local-run/generic; [E5] diagnostics/contract_bounds và log tương ứng; [E6] evidence/github-actions/run-40179312464: CI_EVIDENCE.md, hai ZIP artifacts, selected/physical và PHYSICAL_ANALYSIS.md.

SHA-256 VCD local: 3678ae4b0a8b0a4f1ec920176603d8aa979dffe1e7c93a7bb44db5fcd23a83d5. GDS: 6bb02c420db3967a6b95394187e302c512e116d9a5b9c268b86434cf40f1800a. Chữ ký RTL/TB ở đầu từng tệp trong Phụ lục A/B.

Đối chiếu: 22 tệp đầu vào trùng commit CI; 307 metrics không đổi so với bản lưu trước.
Hình layout giữ bản trước; GDS mới trùng hình học, khác thời gian. Xem verification.json.

Cam kết

Tôi xin cam đoan báo cáo này do tôi tự thực hiện. Toàn bộ số liệu trong báo cáo lấy từ lần chạy trên kho GitHub cá nhân nêu ở trang bìa và có thể đối chiếu được. Tôi hiểu rằng học phần này không thực hiện chế tạo chip, và các kết quả trong báo cáo là kết quả chạy công cụ mô phỏng và tổng hợp, không phải kết quả đo trên silicon.

.

Ngày 27 tháng 10 năm 2026

Người thực hiện

(Ký và ghi rõ họ tên)

Lê Chí Hoàng Cường